



第三章 程序的转换及机器级表示

主要内容



华中科技大学

- ◆ 3.1 程序转换概述
- ◆ 3.2 IA-32指令系统概述
- ◆ 3.3 IA-32常用指令类型及其操作
- ◆ 3.4 C语言的机器级表示
- ◆ 3.5 复杂数据类型的分配和访问
- ◆ 3.6 越界访问和缓冲区攻击





一、数值类型和运算的机器级表示

1. C语言中的整数

- 有符号数: int、short、long
- 无符号数: unsigned int、unsigned short、unsigned long



1. C语言中的整数

注:

- C语言标准规定了各类型最小取值范围，如: int型至少应为16位，取值范围为-32768到32767，而int型数据具体的取值范围则由ABI规范规定。
- 在一个数值的后面加一个“u”或“U”表示无符号数
- C语言标准规定：若运算中同时有无符号和带符号整数，则按无符号整数运算





1. C语言中的整数

例：假定以下关系表达式在32位用补码表示的机器上执行，结果是什么？

关系表达式	有符号/无符号	结果	说明
<code>0 == 0U</code>	无	1	$00...0B = 00...0B$
<code>-1 < 0</code>	有	1	$11...1B(-1) < 00...0B(0)$
<code>-1 < 0U</code>	无	0*	$11...1B(2^{32}-1) > 00...0B(0)$
<code>2147483647 > -2147483647-1</code>	有	1	$011...1B(2^{31}-1) > 100...0B(-2^{31})$
<code>2147483647U > -2147483647-1</code>	无	0*	$011...1B(2^{31}-1) < 100...0B(2^{31})$
<code>2147483647 > (int)2147483648U</code>	有	1*	$011...1B(2^{31}-1) > 100...0B(-2^{31})$
<code>-1 > -2</code>	有	1	$11...1B(-1) > 11...10B(-2)$
<code>(unsigned)-1 > -2</code>	无	1	$11...1B(2^{32}-1) > 11...10B(2^{32}-2)$

带*的结果与常规预想的相反！





1. C语言中的整数

例:

a) 在有些32位系统上, C表达式 $-2147483648 < 2147483647$ 的执行结果为false。Why?

b) 若定义变量 `int i=-2147483648;`, 则 `"i < 2147483647"` 的执行结果为true。Why?

c) 如果将表达式写成 `"-2147483647-1 < 2147483647"`, 则结果会怎样呢? Why?



1. C语言中的整数

a) 在有些32位系统上, C表达式 $-2147483648 < 2147483647$ 的执行结果为false。Why?

在ISO C90标准下, -2147483648 为unsigned int型, 因此
“ $-2147483648 < 2147483647$ ” 按无符号数比较, 10.....0B比
01.....1B大, 结果为false。

在ISO C99标准下, 2147483648 为long long型, 因此
“ $-2147483648 < 2147483647$ ” 按带符号整数比较, 10.....0B比
01.....1B小, 结果为true。





1. C语言中的整数

b) 若定义变量 “`int i=-2147483648;`” , 则 “`i < 2147483647`” 的执行结果为true。Why?

`i < 2147483647` 按int型数比较, 结果为true。

c) 如果将表达式写成 “`-2147483647-1 < 2147483647`” , 则结果会怎样呢? Why?

`-2147483647-1 < 2147483647` 按int型比较, 结果为true。

注: 由C语言中的 “Integer Promotion” (整型提升) 规则决定的。





1. C语言中的整数

例:

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    short  x;
```

```
    unsigned short  y;
```

```
    x= -1;          // x=65535
```

```
    y= -1;          // y=65535
```

```
    printf("%d  %d\n", x, y);
```

```
}
```

结果是: -1 65536



1. C语言中的整数

```
lhb@lhb-VirtualBox:/media/sf_IC3/C03_Transformation/printf_int$ gcc -g -m32 -o p  
rintf_short printf_short.c
```

```
5          short   x;  
6          unsigned short y;  
7          x= -1;  
=> 0x565561b9 <+28>:  movw    $0xffff,-0xc(%ebp)  
  
8          y= -1;  
0x565561bf <+34>:  movw    $0xffff,-0xa(%ebp)  
  
9          printf("%d %d\n", x, y);  
0x565561c5 <+40>:  movzwl -0xa(%ebp),%ecx  
0x565561c9 <+44>:  movswl -0xc(%ebp),%edx  
0x565561cd <+48>:  sub     $0x4,%esp  
0x565561d0 <+51>:  push    %ecx  
0x565561d1 <+52>:  push    %edx  
0x565561d2 <+53>:  lea     -0x1fd0(%eax),%edx  
0x565561d8 <+59>:  push    %edx  
0x565561d9 <+60>:  mov     %eax,%ebx  
0x565561db <+62>:  call    0x56556050 <printf@plt>  
0x565561e0 <+67>:  add     $0x10,%esp  
0x565561e3 <+70>:  mov     $0x0,%eax  
--Type <RET> for more, q to quit, c to continue without paging--  
  
10         }  
0x565561e8 <+75>:  lea     -0x8(%ebp),%esp  
0x565561eb <+78>:  pop     %ecx  
0x565561ec <+79>:  pop     %ebx  
0x565561ed <+80>:  pop     %ebp  
0x565561ee <+81>:  lea     -0x4(%ecx),%esp  
0x565561f1 <+84>:  ret
```





1. C语言中的整数

例:

```
#include <stdio.h>
void main()
{
    short  x;
    unsigned short y;
    x=-1;  y=-1;
    if (x>0)
        printf( "%d positive\n" ,x);
    if (y>0)
        printf( "%d positive\n ",y);
}
```

结果:

65536 positive





1. C语言中的整数

```
7          x=-1;
=> 0x565561ba <+29>:  movw    $0xffff,-0xc(%ebp)

8          y=-1;
0x565561c0 <+35>:  movw    $0xffff,-0xa(%ebp)

9          if (x > 0)
0x565561c6 <+41>:  cmpw    $0x0,-0xc(%ebp)
0x565561cb <+46>:  jle     0x565561e4 <main+71>

10         printf("%d positive\n", x);
0x565561cd <+48>:  movswl  -0xc(%ebp),%eax
0x565561d1 <+52>:  sub     $0x8,%esp
0x565561d4 <+55>:  push    %eax
0x565561d5 <+56>:  lea     -0x1fd0(%ebx),%eax
0x565561db <+62>:  push    %eax
0x565561dc <+63>:  call    0x56556050 <printf@plt>
0x565561e1 <+68>:  add     $0x10,%esp
--Type <RET> for more, q to quit, c to continue without paging--
```

```
11         if (y > 0)
0x565561e4 <+71>:  cmpw    $0x0,-0xa(%ebp)
0x565561e9 <+76>:  je      0x56556202 <main+101>
```

```
12         printf("%d positive\n", y);
0x565561eb <+78>:  movzwl  -0xa(%ebp),%eax
0x565561ef <+82>:  sub     $0x8,%esp
0x565561f2 <+85>:  push    %eax
0x565561f3 <+86>:  lea     -0x1fd0(%ebx),%eax
0x565561f9 <+92>:  push    %eax
0x565561fa <+93>:  call    0x56556050 <printf@plt>
```

原因:

在哪里体现出把一个整数
看作有符号数, 或无符号数?





2.C语言中的浮点数

- C语言中有float和double类型，分别对应IEEE 754单精度浮点数格式和双精度浮点数格式
- long double类型的长度和格式随编译器和处理器类型的不同而有所不同，IA-32中是80位扩展精度格式
- 从int转换为float时，不会发生溢出，但可能有数据被舍入
- 从int或 float转换为double时，因为double的有效位数更多，故能保留精确值
- 从double转换为float和int时，可能发生溢出，此外，由于有效位数变少，故可能被舍入
- 从float 或double转换为int时，因为int没有小数部分，所以数据可能会向0方向被截断





2.C语言中的浮点数

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float heads;
```

```
    while (1) {
```

```
        printf("Please enter a number:");
```

```
        scanf("%f", &heads);
```

```
        printf("%f\r\n", heads);
```

```
    };
```

```
}
```



2.C语言中的浮点数

结果:

```
Please enter a number:61.419997
61.419998
Please enter a number:61.419998
61.419998
Please enter a number:61.419999
61.419998
Please enter a number:61.42
61.419998
Please enter a number:61.420001
61.420002
Please enter a number:█
```

原因:

61.419998和61.420002
是两个可表示数，两者之间相差0.000004。当输入数据是一个不可表示数时，机器将其转换为最邻近的可表示数。



3.数据的宽度和存储

(1) C语言中数值数据类型的宽度

- 高级语言支持多种类型、多种长度的数据

例如，C语言中char类型的宽度为1个字节，可表示一个字符（非数值数据），也可表示一个8位的整数（数值数据）。

不同机器上表示的同一种类型的数据可能宽度不同。





(1) C语言中数值数据类型的宽度

- 必须确定相应的机器级数据表示方式和相应的处理指令
C语言中数值数据类型的宽度 (单位: 字节)

C声明	典型32位 机器	Compaq Alpha 机器
char	1	1
short int	2	2
int	4	4
long int	4	8
char*	4	8
float	4	4
double	8	8

注: Compaq Alpha是一个的64位机器, 即字长为64位。

从表中看出: 同类型数据并不是所有机器都采用相同的宽度, 分配的字节数随机器字长和编译器的不同而不同。





(2) C语言中的类型转换顺序

unsigned long long

↑

long long

↑

unsigned

↑

int

↑

(unsigned)char,short



(2) C语言中的类型转换顺序

例:

```
#include <stdio.h>
int main()
{
    unsigned int a = 1;
    unsigned short b = 1;
    char c = -1;
    int d;
    d = (a > c) ? 1 : 0;
    printf("%d\n", d);
    d = (b > c) ? 1 : 0;
    printf("%d\n", d);
}
```

结果:

0
1





4. C语言中的运算

- C语言中的基本数据类型:

无符号数、带符号整数、浮点数、位串、字符（串）

- 基本运算类型

算术运算、按位逻辑运算、逻辑运算、移位运算、扩展和截断等

- 计算机如何实现高级语言程序中的运算？

- ◆ 将各类表达式编译（转换）为指令序列
- ◆ 计算机直接执行指令来完成运算





4. C语言中的运算

例：C语言赋值语句 “ $f = (g + h) - (i + j);$ ”，反汇编如下：

```
6          f = (g + h) - (i + j);  
=> 0x5655619d <+16>:    mov     -0x14(%ebp),%edx  
      0x565561a0 <+19>:    mov     -0x10(%ebp),%eax  
      0x565561a3 <+22>:    add     %edx,%eax  
      0x565561a5 <+24>:    mov     -0xc(%ebp),%ecx  
      0x565561a8 <+27>:    mov     -0x8(%ebp),%edx  
      0x565561ab <+30>:    add     %ecx,%edx  
      0x565561ad <+32>:    sub     %edx,%eax  
      0x565561af <+34>:    mov     %eax,-0x4(%ebp)  
      0x565561b2 <+37>:    mov     $0x0,%eax
```

需要提供哪些运算类指令才能支持高级语言需求？

如何平衡指令集的实现代价和效率？





(1) 按位逻辑运算和逻辑运算

a) 按位逻辑运算:

- $\&$
- $|$
- $\sim$
- $\wedge$

用途：对位串实现“掩码”（mask）操作或相应的其他处理，主要用于对多媒体数据或状态/控制信息进行处理



b) 逻辑运算

- &&
- ||
- !

用途：用于关系表达式的运算

注意：与按位逻辑运算的差别

- ◆ 符号表示不同
- ◆ 运算过程不同：“按位”对“整体”
- ◆ 结果类型不同：“位串”对“逻辑值”

b) 逻辑运算



华中科技大学

例:

if (x>y || i<100) then ; 条件表达式中的“并”运算

...





(2) 移位运算

- 左移: $x \ll k$
- 右移: $x \gg k$

用途: 提取部分信息; 扩大或缩小数值的2、4、8...倍

注意: C语言中不区分是逻辑移位还是算术移位, 编译器根据x的类型确定

- ◆ 无符号数: 逻辑左移, 逻辑右移
- ◆ 有符号数: 算术左移, 算术右移



華中科技大學

(2) 移位运算

例：从16位数据y中提取高位字节

$y = y \gg 8$





(3) 位扩展运算和位截断运算

C语言中没有专门操作运算符，编译器根据类型转换前后数据的长短，确定是扩展还是截断。

- 扩展：短转长
 - ◆ 无符号数：0扩展，前面补0
 - ◆ 带符号整数：符号扩展，前面补符
- 截断：长转短
 - ◆ 强行将高位丢弃

用途：

类型转换时可能需要数据扩展或截断





(3) 位扩展运算和位截断运算

例：整数 si , usi , i , ui 的十进制和十六进制值是什么？

```
short si = -32768;
```

```
unsigned short usi = si;
```

```
int i = si;
```

```
unsigned ui = usi;
```

结果：

```
si = -32768; 0x8000
```

```
usi = 32768; 0x8000
```

```
i = -32768; 0xffff8000
```

```
ui = 32768; 0x00008000
```



(3) 位扩展运算和位截断运算

例：整数i和j是否相等？

```
int i = 32768;
```

```
short si = (short)i;
```

```
int j = si;
```

结果：

不相等！

```
i = 32768; 0x00008000
```

```
si = -32768; 0x8000
```

```
j = -32768; 0xffff8000
```

原因：

对i截断时发生了溢出，即
：32768截断为16位数时，
因其超出16位能表示的最大
值，故无法截断为正确的16
位数！





(4) 整数算术运算

C语言中的算术运算，包括无符号数、有符号数、浮点数中的加、减、乘、除运算。

a) C语言整数实现与汇编语言的差异

- 整数类型添加了符号信息
- 运算符取消了类型信息，运算时可能跨类型
- 运算时，取消了标志寄存器的直接观察窗口
- 运算时，可能存在隐式的类型转化（类型扩展或截断）





b) 无符号数加法溢出判断

如何用程序判断一个无符号数相加没有发生溢出？

$$\text{result} = \begin{cases} x+y & (x+y < 2^n) \\ x+y-2^n & (2^n \leq x+y < 2^{n+1}) \end{cases}$$

因此，发生溢出时，一定满足：

result < x 且 result < y

(否则，若 $x + y - 2^n \geq x$ ，则 $y \geq 2^n$ ，不成立。同理可证 $\text{result} < y$ 。)



b) 无符号数加法溢出判断

无符号数加法溢出判断函数如下：

```
int uadd_ok(unsigned x, unsigned y)
{
    unsigned sum = x + y;
    return su >= x;
}
```



c) 有符号数加法溢出判断

如何用程序判断一个带符号整数相加没有发生溢出？

$$\text{result} = \begin{cases} x+y-2^n & (2^{n-1} \leq x+y) & \text{正溢出} \\ x+y & (-2^{n-1} \leq x+y < 2^{n-1}) & \text{正常} \\ x+y+2^n & (x+y < -2^{n-1}) & \text{负溢出} \end{cases}$$



c) 有符号数加法溢出判断

有符号数加法溢出判断函数如下:

```
int tadd_ok(int x, int y) {  
    int sum = x+y;  
    int neg_over = x < 0 && y < 0 && sum >= 0;  
    int pos_over = x >= 0 && y >= 0 && sum <  
0;  
    return !neg_over && !pos_over;  
}
```





c) 有符号数加法溢出判断

例:

以下程序检查带符号整数相减是否溢出有没有问题?

```
int tsub_ok(int x, int y) {  
    return tadd_ok(x, -y);  
}
```

考虑 $y = 0x80000000$ 时的情况, 函数判断出错



d) 整数的乘运算

在**C语言**中，参加运算的两个操作数的类型和结果的**类型必须一致**，如果**不一致则会先转换为一致的数据类型**再进行计算。

通常，高级语言中两个 n 位整数相乘得到的结果通常也是一个 n 位整数，也即，结果只取 $2n$ 位乘积中的低 n 位。

乘法指令的操作数长度为 n ，而乘积长度为 $2n$ 。硬件保留 $2n$ 位乘积，即汇编指令的乘积可达 $2n$ 位，可供编译器使用。

乘法指令可生成溢出标志，编译器也可使用 $2n$ 位乘积来判断是否溢出！如果程序不采用防止溢出的措施，且编译器也不生成用于溢出处理的代码，就会发生一些由于整数溢出而带来的问题。



d) 整数的乘运算

例：以下程序存在什么漏洞，引起该漏洞的原因是什么？

```
int copy_array(int *array, int count)
{
    int i;
    /* 在堆区申请一块内存 */
    int *myarray = (int *) malloc(count*sizeof(int));
    if (myarray == NULL)
        return -1;
    for (i = 0; i < count; i++)
        myarray[i] = array[i];
    return count;
}
```





d) 整数的乘运算

当参数count很大时，则 $\text{count} * \text{sizeof}(\text{int})$ 会溢出。如 $\text{count} = 2^{30} + 1$ 时， $\text{count} * \text{sizeof}(\text{int}) = 4$ 。因此，堆 (heap) 中大量数据被破坏！

攻击者可构造特殊参数来触发整数溢出，以一段预设信息覆盖一个已分配的堆缓冲区，造成远程服务器崩溃或者改变内存数据并执行任意代码

2002年，Sun Microsystems公司的RPC XDR库带的xdr_array函数发生整数溢出漏洞，攻击者可利用该漏洞从远程或本地获取root权限。





e) 整数的除法运算

例:

代码段一:

```
int a = 0x80000000;
```

```
int b = a / -1;
```

```
printf("%d\n", b);
```

运行结果为-2147483648

代码段二:

```
int a = 0x80000000;
```

```
int b = -1;
```

```
int c = a / b;
```

```
printf("%d\n", c);
```

运行结果为“Floating point exception”，显然CPU检测到了异常



e) 整数的除法运算

例:

代码段一:

```
int a = 0x80000000;
```

```
int b = a / -1;
```

```
printf("%d\n", b);
```

运行结果为-2147483648

代码段二:

```
int a = 0x80000000;
```

```
int b = -1;
```

```
int c = a / b;
```

```
printf("%d\n", c);
```

运行结果为“Floating point exception”，显然CPU检测到了异常



(5) 浮点数的运算

a) 浮点数的除0运算

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 1;
```

```
    int b = 0;
```

```
    printf("Division by zero: %d\n", a/b);
```

```
    return 0;
```

```
}
```

结果：整数除0会发生异常！





a) 浮点数的除0运算

```
#include <stdio.h>
int main()
{
    double a = 1.0;
    double b = -1.0;
    double c = 0.0;
    printf("Division by zero: %f %f\n", a/c, b/c);
    return 0;
}
```

结果：浮点数除0不会发生异常

浮点数除0，结果为正无穷大或负无穷大





b) 浮点数的比较运算

对于以下给定的关系表达式，判断是否永真。

表达式	结果
$x == (\text{int})(\text{float})\ x$	否
$x == (\text{int})(\text{double})\ x$	是
$f == (\text{float})(\text{double})\ f$	是
$d == (\text{float})\ d$	否
$f == -(-f);$	是
$2/3 == 2/3.0$	否
$d < 0.0 \Rightarrow ((d*2) < 0.0)$	是
$d > f \Rightarrow -f > -d$	是
$d * d \geq 0.0$	是
$x * x \geq 0$	否
$(d+f)-d == f$	否



c) 爱国者导弹定位错误的例子

例：1991年2月25日，海湾战争中，美国在沙特阿拉伯达摩地区设置的爱国者导弹拦截伊拉克的飞毛腿导弹失败，致使飞毛腿导弹击中了一个美军军营，杀死了美军28名士兵。其原因是由于爱国者导弹系统时钟内的一个软件错误造成的，引起这个软件错误的原因是浮点数的精度问题。



c) 爱国者导弹定位错误的例子

爱国者导弹系统中有一内置时钟，用计数器实现，每隔0.1秒计数一次。程序用0.1的一个24位定点二进制小数x来乘以计数值作为以秒为单位的时间

这个x的机器数是多少呢？

0.1的二进制表示是一个无限循环序列：0.00011[0011]...,
 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\text{B}$ 。显然，x是0.1的近似表示

。

$$0.1 - x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ [1100]... -$$

$$0.000\ 1100\ 1100\ 1100\ 1100\ 1100\text{B},$$

$$= 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ [1100]... \text{B}$$

$$= 2^{-20} \times 0.1$$

$$\approx 9.54 \times 10^{-8}$$

这就是机器值与真值之间的误差！





华中科技大学

c) 爱国者导弹定位错误的例子

已知在爱国者导弹准备拦截飞毛腿导弹之前，已经连续工作了100小时，飞毛腿的速度大约为2000米/秒，则由于时钟计算误差而导致的距离误差是多少？

100小时相当于计数了 $100 \times 60 \times 60 \times 10 = 36 \times 10^5$ 次，因而导弹的时钟已经偏差了

$9.54 \times 10^{-8} \times 36 \times 10^5 \approx 0.343$ 秒。

因此，距离误差是 $2000 \times 0.343 \text{秒} \approx 687 \text{米}$





c) 爱国者导弹定位错误的例子

小故事：实际上，以色列方面已经发现了这个问题并于1991年2月11日知会了美国陆军及爱国者计划办公室（软件制造商）。以色列方面建议重新启动爱国者系统的电脑作为暂时解决方案，可是美国陆军方面却不知道每次需要间隔多少时间重新启动系统一次。1991年2月16日，制造商向美国陆军提供了更新软件，但这个软件最终却在飞毛腿导弹击中军营后的一天才运抵部队。



c) 爱国者导弹定位错误的例子

- 若x用float型表示, 则x的机器数是什么? 0.1与x的偏差是多少? 系统运行100小时后的时钟偏差是多少? 在飞毛腿速度为2000米/秒的情况下, 预测的距离偏差为多少?

$0.1 = 0.00011[0011]B = +1.10011001100110011001100B \times 2^{-4}$, 故x的机器数为0011110111001100110011001100

Float型仅24位有效位数, 后面的有效位全被丢弃, 故x与0.1之间的误差为: $|x - 0.1| = 0.0000000000000000000000001100[1100]...B$ 。这个值等于 $2^{-24} \times 0.1 = 5.96 \times 10^{-9}$ 。100小时后时钟偏差 $5.96 \times 10^{-9} \times 36 \times 10^5 = 0.0215$ 秒。距离偏差 $0.0215 \times 2000 = 43$ 米。比爱国者导弹系统精确约16倍。



c) 爱国者导弹定位错误的例子

- 若用32位二进制定点小数 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ B$ 表示0.1, 则误差比用float表示误差更大还是更小?

当 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ B$ 时, 与0.1之间的误差约为: $|x-0.1|=0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 00\ 1100\ [1100]...B$ 。这个值等于 $2^{-30} \times 0.1 = 9.31 \times 10^{-11}$ 。100小时后时钟偏差 $9.31 \times 10^{-11} \times 36 \times 10^5 = 0.000335$ 秒。预测的距离偏差仅为 $0.000335 \times 2000 = 0.67$ 米。



二、选择语句的机器级表示

1.if-else语句的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int flag;
```

```
    int x = 3;
```

```
    int y = -1;
```

```
    if (x > 0 || y > 0)
```

```
        flag = 1;
```

```
    else
```

```
        flag = 0;
```

```
}
```

```
    printf("flag = %d \n", flag);  
    return 0;
```



1.if-else语句的机器级表示

```
5          int flag;
6          int x = 3;
=> 0x565561b9 <+28>:    movl    $0x3,-0x10(%ebp)

7          int y = -1;
0x565561c0 <+35>:    movl    $0xffffffff,-0xc(%ebp)

8          if (x > 0 || y > 0)
0x565561c7 <+42>:    cmpl    $0x0,-0x10(%ebp)
0x565561cb <+46>:    jg     0x565561d3 <main+54>
```



1.if-else语句的机器级表示

```
0x565561cd <+48>:    cmpl    $0x0, -0xc(%ebp)
0x565561d1 <+52>:    jle     0x565561dc <main+63>

9                     flag = 1;
0x565561d3 <+54>:    movl    $0x1, -0x14(%ebp)
0x565561da <+61>:    jmp     0x565561e3 <main+70>

10                    else
11                     flag = 0;
0x565561dc <+63>:    movl    $0x0, -0x14(%ebp)

12
13                    printf("flag = %d \n", flag);
0x565561e3 <+70>:    sub     $0x8, %esp
0x565561e6 <+73>:    push    -0x14(%ebp)
0x565561e9 <+76>:    lea     -0x1fd0(%eax), %edx
0x565561ef <+82>:    push    %edx
0x565561f0 <+83>:    mov     %eax, %ebx
0x565561f2 <+85>:    call    0x56556050 <printf@plt>
0x565561f7 <+90>:    add     $0x10, %esp
```



2.switch的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x = 3;
```

```
    int y = -1;
```

```
    int z;
```

```
    int i = 1;
```

```
    char c;
```

```
    c = getchar();
```

```
    switch (c) {
```

```
        case '+':
```

```
        case 'a':
```

```
            z = x + y;
```

```
            break;
```

```
        case '-':
```

```
        case 's':
```

```
            z = x - y;
```

```
            break;
```

```
        default:
```

```
            z = 0;
```

```
    }
```

```
    printf(" %d %c %d = %d \n", x, c,  
y, z);
```

```
    return 0;
```

```
}
```



2.switch的机器级表示

```
9      char c;
10     c = getchar();
      0x565561df <+50>:  call    0x56556060 <getchar@plt>
      0x565561e4 <+55>:  mov     %al,-0x19(%ebp)

11     switch (c) {
      0x565561e7 <+58>:  movsbl  -0x19(%ebp),%eax
      0x565561eb <+62>:  cmp     $0x73,%eax
      0x565561ee <+65>:  je      0x56556218 <main+107>
      0x565561f0 <+67>:  cmp     $0x73,%eax
      0x565561f3 <+70>:  jg      0x56556223 <main+118>
      0x565561f5 <+72>:  cmp     $0x61,%eax
      0x565561f8 <+75>:  je      0x5655620b <main+94>
      0x565561fa <+77>:  cmp     $0x61,%eax
      0x565561fd <+80>:  jg      0x56556223 <main+118>
      0x565561ff <+82>:  cmp     $0x2b,%eax
      0x56556202 <+85>:  je      0x5655620b <main+94>
      0x56556204 <+87>:  cmp     $0x2d,%eax
      0x56556207 <+90>:  je      0x56556218 <main+107>
      0x56556209 <+92>:  jmp     0x56556223 <main+118>
```





2.switch的机器级表示

```
12         case '+':
13         case 'a':
--Type <RET> for more, q to quit, c to continue without paging--
14         z = x + y;
0x5655620b <+94>:  mov     -0x14(%ebp),%edx
0x5655620e <+97>:  mov     -0x10(%ebp),%eax
0x56556211 <+100>: add     %edx,%eax
0x56556213 <+102>: mov     %eax,-0x18(%ebp)

15         break;
0x56556216 <+105>: jmp     0x5655622a <main+125>

16         case '-':
17         case 's':
18         z = x - y;
0x56556218 <+107>: mov     -0x14(%ebp),%eax
0x5655621b <+110>: sub     -0x10(%ebp),%eax
0x5655621e <+113>: mov     %eax,-0x18(%ebp)

19         break;
0x56556221 <+116>: jmp     0x5655622a <main+125>

20         default:
21         z = 0;
0x56556223 <+118>: movl    $0x0,-0x18(%ebp)
```





2.switch的机器级表示

```
22         }
23         printf(" %d %c %d = %d \n", x, c, y, z);
0x5655622a <+125>: movsbl -0x19(%ebp),%eax
0x5655622e <+129>: sub    $0xc,%esp
0x56556231 <+132>: push  -0x18(%ebp)
0x56556234 <+135>: push  -0x10(%ebp)
0x56556237 <+138>: push  %eax
0x56556238 <+139>: push  -0x14(%ebp)
0x5655623b <+142>: lea    -0x1fcc(%ebx),%eax
0x56556241 <+148>: push  %eax
0x56556242 <+149>: call  0x56556050 <printf@plt>
0x56556247 <+154>: add    $0x20,%esp
```

用到的字符的ASCII码

:

‘+’: 0x2b

‘-’: 0x2d

‘a’: 0x61

‘s’: 0x73





2.switch的机器级表示

例：使用跳转表实现switch语句

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int result = 0;
```

```
    int a = 12;
```

```
    int c = 0;
```

```
    int b = 10;
```

```
    switch (a)
```

```
    {
```

```
        case 15:
```

```
            c = b & 0x0f;
```

```
        case 10:
```

```
            result = c + 50;
```

```
        break;
```

```
        case 12:
```

```
        case 17:
```

```
            result = b + 50;
```

```
            break;
```

```
        case 14:
```

```
            result = b;
```

```
            break;
```

```
        default:
```

```
            result = a;
```

```
    }
```

```
    printf("%d\n", result);
```

```
    return 0;
```

```
}
```



2.switch的机器级表示



华中科技大学

例：使用跳转表实现switch语句

每个分支的条件值为一个整数，因此可以用条件值和分支入口地址的对应关系，生成一个跳转表。通过输入变量在跳转表中进行查找，得到跳转的目的地址。





2.switch的机器级表示

```
9      switch (a)
    0x565561d5 <+56>:  mov     -0x10(%ebp),%edx
    0x565561d8 <+59>:  sub     $0xa,%edx
=> 0x565561db <+62>:  cmp     $0x7,%edx
    0x565561de <+65>:  ja      0x56556215 <main+120>
    0x565561e0 <+67>:  shl     $0x2,%edx
    0x565561e3 <+70>:  mov     -0x1fcc(%edx,%eax,1),%edx
    0x565561ea <+77>:  add     %eax,%edx
    0x565561ec <+79>:  jmp     *%edx

10     {
11         case 15:
12             c = b & 0x0f;
    0x565561ee <+81>:  mov     -0xc(%ebp),%edx
    0x565561f1 <+84>:  and     $0xf,%edx
    0x565561f4 <+87>:  mov     %edx,-0x14(%ebp)

13         case 10:
14             result = c + 50;
    0x565561f7 <+90>:  mov     -0x14(%ebp),%edx
    0x565561fa <+93>:  add     $0x32,%edx
```





2.switch的机器级表示

```
0x565561fd <+96>:    mov    %edx, -0x18(%ebp)

15                break;
0x56556200 <+99>:    jmp     0x5655621b <main+126>

16                case 12:
17                case 17:
18                result = b + 50;
0x56556202 <+101>:    mov     -0xc(%ebp), %edx
0x56556205 <+104>:    add     $0x32, %edx
0x56556208 <+107>:    mov     %edx, -0x18(%ebp)

19                break;
0x5655620b <+110>:    jmp     0x5655621b <main+126>

20                case 14:
21                result = b;
0x5655620d <+112>:    mov     -0xc(%ebp), %edx
0x56556210 <+115>:    mov     %edx, -0x18(%ebp)

22                break;
0x56556213 <+118>:    jmp     0x5655621b <main+126>
```





2.switch的机器级表示

```
23          default:
24          result = a;
0x56556215 <+120>:  mov    -0x10(%ebp),%edx
0x56556218 <+123>:  mov    %edx,-0x18(%ebp)
25          }
```

当执行到EIP = 0x565561d5时，EAX = 0x56558fd8。则由EIP = 0x565561e3处的指令可知，跳转表的首址为 EAX - 0x1fcc = 0x56558fd8 - 0x1fcc = 0x5655700c。



2.switch的机器级表示

跳转表按“相对”的条件值存放的“相对”的分支入口地址。

条件值和入口地址的对应表如下：

条件值	跳转表索引值	分支入口地址	跳转表项(=入口地址-EAX)
10	0	0x565561f7	0xffffd21f
12	2	0x56556202	0xffffd22a
14	4	0x5655620d	0xffffd235
15	5	0x565561ee	0xffffd216
17	7	0x56556202	0xffffd22a
default	/	0x56556215	0xffffd23d



2.switch的机器级表示

条件值	跳转表索引值	分支入口地址	跳转表项(=入口地址-EAX)
10	0	0x565561f7	0xffffd21f
12	2	0x56556202	0xffffd22a
14	4	0x5655620d	0xffffd235
15	5	0x565561ee	0xffffd216
17	7	0x56556202	0xffffd22a
default	/	0x56556215	0xffffd23d

```
(gdb) x/16 0x5655700c
```

```
0x5655700c:  0xffffd21f  0xffffd23d  0xffffd22a  0xffffd23d
0x5655701c:  0xffffd235  0xffffd216  0xffffd23d  0xffffd22a
0x5655702c:  0x3b031b01  0x00000030  0x00000005  0xfffff004
0x5655703c:  0x00000060  0xfffff034  0x00000084  0xfffff044
```





3. 条件表达式的机器级表示

```
#include <stdio.h>

int main()
{
    int a;
    int x;
    scanf("%d", &a);
    x = a > 0 ? a : a + 100;
    printf("x = %d \n", x);
    return 0;
}
```

```
7          scanf("%d", &a);
=> 0x565561e5 <+40>:  sub    $0x8,%esp
    0x565561e8 <+43>:  lea     -0x14(%ebp),%eax
    0x565561eb <+46>:  push   %eax
    0x565561ec <+47>:  lea     -0x1fc8(%ebx),%eax
```



3. 条件表达式的机器级表示

```
0x565561f2 <+53>:  push    %eax
0x565561f3 <+54>:  call    0x56556070 <__isoc99_scanf@plt>
0x565561f8 <+59>:  add     $0x10,%esp
```

```
8          x = a > 0 ? a : a + 100;
0x565561fb <+62>:  mov     -0x14(%ebp),%eax
0x565561fe <+65>:  test    %eax,%eax
0x56556200 <+67>:  jg      0x5655620a <main+77>
0x56556202 <+69>:  mov     -0x14(%ebp),%eax
0x56556205 <+72>:  add     $0x64,%eax
0x56556208 <+75>:  jmp     0x5655620d <main+80>
0x5655620a <+77>:  mov     -0x14(%ebp),%eax
0x5655620d <+80>:  mov     %eax,-0x10(%ebp)
```

```
9          printf("x = %d \n", x);
0x56556210 <+83>:  sub     $0x8,%esp
0x56556213 <+86>:  push    -0x10(%ebp)
0x56556216 <+89>:  lea     -0x1fc5(%ebx),%eax
0x5655621c <+95>:  push    %eax
0x5655621d <+96>:  call    0x56556050 <printf@plt>
0x56556222 <+101>: add     $0x10,%esp
```





三、循环结构的机器级表示

1.do-while循环的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 1;
```

```
    int result = 0;
```

```
    do
```

```
    {
```

```
        result += i;
```

```
        i++;
```

```
    }
```

```
while (i <= 10);
```

```
printf("result = %d \n", result);
```

```
return 0;
```

```
}
```



1.do-while循环的机器级表示

```
4          int i = 1;
=> 0x565561b9 <+28>:    movl    $0x1,-0x10(%ebp)

5          int result = 0;
0x565561c0 <+35>:    movl    $0x0,-0xc(%ebp)

6          do
7          {
8              result += i;
0x565561c7 <+42>:    mov     -0x10(%ebp),%edx
0x565561ca <+45>:    add     %edx,-0xc(%ebp)

9              i++;
0x565561cd <+48>:    addl    $0x1,-0x10(%ebp)
```



1.do-while循环的机器级表示

```
10         }
11         while (i <= 10);
0x565561d1 <+52>:    cmpl    $0xa,-0x10(%ebp)
0x565561d5 <+56>:    jle     0x565561c7 <main+42>

12
13         printf("result = %d \n", result);
0x565561d7 <+58>:    sub     $0x8,%esp
0x565561da <+61>:    push   -0xc(%ebp)
0x565561dd <+64>:    lea     -0x1fd0(%eax),%edx
0x565561e3 <+70>:    push   %edx
0x565561e4 <+71>:    mov     %eax,%ebx
0x565561e6 <+73>:    call    0x56556050 <printf@plt>
0x565561eb <+78>:    add     $0x10,%esp
```



2.while语句的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i = 1;
```

```
    int n = 100;
```

```
    int result = 0;
```

```
    while (i <= n)
```

```
    {
```

```
        result += i;
```

```
        i++;
```

```
    }
```

```
    printf("result = %d \n", result);
```

```
    return 0;
```

```
}
```



2.while语句的机器级表示

```
4          int i = 1;
=> 0x565561b9 <+28>:    movl    $0x1, -0x14(%ebp)

5          int n = 100;
0x565561c0 <+35>:    movl    $0x64, -0xc(%ebp)

6          int result = 0;
0x565561c7 <+42>:    movl    $0x0, -0x10(%ebp)

7          while (i <= n)
0x565561ce <+49>:    jmp     0x565561da <main+61>
0x565561da <+61>:    mov     -0x14(%ebp),%edx
0x565561dd <+64>:    cmp     -0xc(%ebp),%edx
0x565561e0 <+67>:    jle     0x565561d0 <main+51>
```



2.while语句的机器级表示

```
8      {
9          result += i;
0x565561d0 <+51>:  mov     -0x14(%ebp),%edx
0x565561d3 <+54>:  add     %edx,-0x10(%ebp)

10         i++;
0x565561d6 <+57>:  addl    $0x1,-0x14(%ebp)

11     }

12
13     printf("result = %d \n", result);
0x565561e2 <+69>:  sub     $0x8,%esp
0x565561e5 <+72>:  push    -0x10(%ebp)
0x565561e8 <+75>:  lea     -0x1fd0(%eax),%edx
0x565561ee <+81>:  push    %edx
0x565561ef <+82>:  mov     %eax,%ebx
0x565561f1 <+84>:  call    0x56556050 <printf@plt>
0x565561f6 <+89>:  add     $0x10,%esp
```



3.for语句的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n = 100;
```

```
    int result = 0;
```

```
    for (int i = 1; i <= n; i++)
```

```
        result += i;
```

```
    printf("result = %d \n", result);
```

```
    return 0;
```

```
}
```





3.for语句的机器级表示

```
4          int n = 100;
=> 0x565561b9 <+28>:    movl    $0x64,-0xc(%ebp)

5          int result = 0;
0x565561c0 <+35>:    movl    $0x0,-0x14(%ebp)

6          for (int i = 1; i <= n; i++)
0x565561c7 <+42>:    movl    $0x1,-0x10(%ebp)
0x565561ce <+49>:    jmp     0x565561da <main+61>
0x565561d6 <+57>:    addl    $0x1,-0x10(%ebp)
0x565561da <+61>:    mov     -0x10(%ebp),%edx
0x565561dd <+64>:    cmp     -0xc(%ebp),%edx
0x565561e0 <+67>:    jle     0x565561d0 <main+51>
```



3.for语句的机器级表示

```
7          result += i;
0x565561d0 <+51>:  mov    -0x10(%ebp),%edx
0x565561d3 <+54>:  add    %edx,-0x14(%ebp)

8          printf("result = %d \n", result);
0x565561e2 <+69>:  sub    $0x8,%esp
0x565561e5 <+72>:  push   -0x14(%ebp)
0x565561e8 <+75>:  lea    -0x1fd0(%eax),%edx
0x565561ee <+81>:  push   %edx
0x565561ef <+82>:  mov    %eax,%ebx
0x565561f1 <+84>:  call   0x56556050 <printf@plt>
0x565561f6 <+89>:  add    $0x10,%esp
```



3.for语句的机器级表示

```
9          return 0;
0x565561f9 <+92>:  mov    $0x0,%eax

10      }
0x565561fe <+97>:  lea    -0x8(%ebp),%esp
0x56556201 <+100>: pop    %ecx
0x56556202 <+101>: pop    %ebx
0x56556203 <+102>: pop    %ebp
0x56556204 <+103>: lea    -0x4(%ecx),%esp
0x56556207 <+106>: ret
```



3.for语句的机器级表示

例:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char  buf1[20];
```

```
    char  buf2[20];
```

```
    int   i;
```

```
    scanf("%s", buf1);
```

```
    for (i = 0; i < 20; i++)
```

```
        buf2[i] = buf1[i];
```

```
    printf("%s\n", buf2);
```

```
    return 0;
```

```
}
```



3.for语句的机器级表示

```
8      for (i = 0; i < 20; i++)
0x565561fb <+62>:    movl    $0x0, -0x38(%ebp)
0x56556202 <+69>:    jmp     0x5655621d <main+96>
0x56556219 <+92>:    addl    $0x1, -0x38(%ebp)
0x5655621d <+96>:    cmpl    $0x13, -0x38(%ebp)
0x56556221 <+100>:   jle     0x56556204 <main+71>
```

```
9      buf2[i] = buf1[i];
0x56556204 <+71>:    lea     -0x34(%ebp), %edx
0x56556207 <+74>:    mov     -0x38(%ebp), %eax
0x5655620a <+77>:    add     %edx, %eax
0x5655620c <+79>:    movzbl  (%eax), %eax
0x5655620f <+82>:    lea     -0x20(%ebp), %ecx
0x56556212 <+85>:    mov     -0x38(%ebp), %edx
0x56556215 <+88>:    add     %ecx, %edx
0x56556217 <+90>:    mov     %al, (%edx)
```



4. 编译优化

gcc提供了近百种编译优化选项，对

- 编译时间
- 目标文件长度
- 执行效率

三个方面进行取舍和平衡，进行优化。



4. 编译优化

gcc提供了O0、O1、O2、O3四种优化级别。这四种优化级别，按递增的方式，包含了不同的上述优化选项，简化的编译优化的使用难度。

- -O0：不做任何优化
- -O1：尝试减小生成代码的尺寸，以及缩短执行时间。但不执行需要占用大量编译时间的优化
- -O2：在O1的基础上，进行更多的优化，包括大部分较消耗编译时间的优化
- -O3：在O2的基础上，进行更进一步的优化





4.编译优化

例：对switch.c使用O1编译优化选项。

编译命令为：gcc -g -m32 -O1 -o switch switch.c

可执行程序反汇编如下：





4. 编译优化

```
0x565561ad <+0>:    lea     0x4(%esp), %ecx
0x565561b1 <+4>:    and     $0xffffffff0, %esp
0x565561b4 <+7>:    push    -0x4(%ecx)
0x565561b7 <+10>:   push    %ebp
0x565561b8 <+11>:   mov     %esp, %ebp
0x565561ba <+13>:   push    %ebx
0x565561bb <+14>:   push    %ecx
0x565561bc <+15>:   call    0x565560b0 <__x86.get_pc_thunk.bx>
0x565561c1 <+20>:   add     $0x2e0f, %ebx
0x565561c7 <+26>:   sub     $0xc, %esp
0x565561ca <+29>:   mov     0x24(%ebx), %eax
0x565561d0 <+35>:   push    (%eax)
0x565561d2 <+37>:   call    0x56556060 <getc@plt>
=> 0x565561d7 <+42>:   movsbl  %al, %ecx
0x565561da <+45>:   add     $0x10, %esp
0x565561dd <+48>:   cmp     $0x61, %al
0x565561df <+50>:   je      0x5655622d <main+128>
0x565561e1 <+52>:   jg      0x56556220 <main+115>
0x565561e3 <+54>:   mov     $0x2, %edx
0x565561e8 <+59>:   cmp     $0x2b, %al
0x565561ea <+61>:   je      0x565561f7 <main+74>
0x565561ec <+63>:   cmp     $0x2d, %al
0x565561ee <+65>:   sete    %dl
0x565561f1 <+68>:   movzbl  %dl, %edx
0x565561f4 <+71>:   shl     $0x2, %edx
0x565561f7 <+74>:   sub     $0x8, %esp
0x565561fa <+77>:   push    %edx
0x565561fb <+78>:   push    $0xffffffff
```





4. 编译优化

```
0x565561fd <+80>:  push    %ecx
0x565561fe <+81>:  push    $0x3
0x56556200 <+83>:  lea     -0x1fc8(%ebx), %eax
0x56556206 <+89>:  push    %eax
0x56556207 <+90>:  push    $0x1
0x56556209 <+92>:  call    0x56556050 <__printf_chk@plt>
0x5655620e <+97>:  add     $0x20, %esp
0x56556211 <+100>:  mov     $0x0, %eax
0x56556216 <+105>:  lea     -0x8(%ebp), %esp
0x56556219 <+108>:  pop     %ecx
0x5655621a <+109>:  pop     %ebx
0x5655621b <+110>:  pop     %ebp
0x5655621c <+111>:  lea     -0x4(%ecx), %esp
0x5655621f <+114>:  ret
0x56556220 <+115>:  cmp     $0x73, %al
0x56556222 <+117>:  sete    %dl
0x56556225 <+120>:  movzbl  %dl, %edx
0x56556228 <+123>:  shl     $0x2, %edx
0x5655622b <+126>:  jmp     0x565561f7 <main+74>
0x5655622d <+128>:  mov     $0x2, %edx
0x56556232 <+133>:  jmp     0x565561f7 <main+74>
```





4. 编译优化

例：对for.c使用O1编译优化选项。

```
0x565561b7 <+26>:    mov     $0x64,%eax
0x565561bc <+31>:    sub     $0x1,%eax
0x565561bf <+34>:    jne     0x565561bc <main+31>
0x565561c1 <+36>:    sub     $0x4,%esp
0x565561c4 <+39>:    push    $0x13ba
0x565561c9 <+44>:    lea     -0x1fd0(%ebx),%eax
0x565561cf <+50>:    push    %eax
0x565561d0 <+51>:    push    $0x1
0x565561d2 <+53>:    call    0x56556050 <__printf_chk@plt>
0x565561d7 <+58>:    add     $0x10,%esp
```



4. 编译优化

例：对copy_a.c使用O1编译优化选项

```
8      for (i = 0; i < 20; i++)
0x565561f8 <+59>:    mov     $0x0,%eax
0x56556206 <+73>:    add     $0x1,%eax
0x56556209 <+76>:    cmp     $0x14,%eax
0x5655620c <+79>:    jne     0x565561fd <main+64>

9          buf2[i] = buf1[i];
0x565561fd <+64>:    movzbl  -0x34(%ebp,%eax,1),%edx
0x56556202 <+69>:    mov     %dl,-0x20(%ebp,%eax,1)

10         printf("%s\n", buf2);
11         return 0;
```

将循环控制变量i，使用EAX代替。减少了内存访问，从而提高了执行效率。





5. 循环结构与递归的比较

```
#include <stdio.h>

int iter_sum(int n)
{
    int result;
    if (n <= 0)
        result = 0;
    else
        result = n + iter_sum(n - 1);
    return result;
}
```

```
int main()
{
    int sum = iter_sum(10);
    printf("%d\n", sum);
    return 0;
}
```



5. 循环结构与递归的比较

```
4      {
    0x5655619d <+0>:    push    %ebp
    0x5655619e <+1>:    mov     %esp,%ebp
    0x565561a0 <+3>:    sub     $0x18,%esp
    0x565561a3 <+6>:    call   0x5655622c <__x86.get_pc_thunk.ax>
    0x565561a8 <+11>:   add     $0x2e30,%eax

5          int result;
6          if (n <= 0)
=> 0x565561ad <+16>:   cmpl    $0x0,0x8(%ebp)
    0x565561b1 <+20>:   jg      0x565561bc <iter_sum+31>

7              result = 0;
    0x565561b3 <+22>:   movl    $0x0,-0xc(%ebp)
    0x565561ba <+29>:   jmp     0x565561d6 <iter_sum+57>

8          else
9              result = n + iter_sum(n - 1);
    0x565561bc <+31>:   mov     0x8(%ebp),%eax
    0x565561bf <+34>:   sub     $0x1,%eax
    0x565561c2 <+37>:   sub     $0xc,%esp
    0x565561c5 <+40>:   push    %eax
    0x565561c6 <+41>:   call   0x5655619d <iter_sum>
    0x565561cb <+46>:   add     $0x10,%esp
    0x565561ce <+49>:   mov     0x8(%ebp),%edx
    0x565561d1 <+52>:   add     %edx,%eax
```





5. 循环结构与递归的比较

```
0x565561d3 <+54>:  mov    %eax,-0xc(%ebp)

10          return  result;
0x565561d6 <+57>:  mov    -0xc(%ebp),%eax

11      }
0x565561d9 <+60>:  leave
0x565561da <+61>:  ret
```

每次递归调用都会增加一个栈帧，所以空间开销很大。当n很大时会发生栈溢出！



5. 循环结构与递归的比较

```
#include <stdio.h>
int main()
{
    int n = 10;
    int i;
    int result = 0;
    for (i=1; i <= n; i++)
        result += i;
    printf("%d\n", result);
    return 0;
}
```



5. 循环结构与递归的比较

```
5          int n = 10;
=> 0x565561b9 <+28>:    movl    $0xa, -0xc(%ebp)

6          int i;
7          int result = 0;
0x565561c0 <+35>:    movl    $0x0, -0x10(%ebp)

8
9          for (i=1; i <= n; i++)
0x565561c7 <+42>:    movl    $0x1, -0x14(%ebp)
0x565561ce <+49>:    jmp     0x565561da <main+61>
0x565561d6 <+57>:    addl    $0x1, -0x14(%ebp)
0x565561da <+61>:    mov     -0x14(%ebp), %edx
0x565561dd <+64>:    cmp     -0xc(%ebp), %edx
0x565561e0 <+67>:    jle     0x565561d0 <main+51>

10         result += i;
0x565561d0 <+51>:    mov     -0x14(%ebp), %edx
0x565561d3 <+54>:    add     %edx, -0x10(%ebp)
```



5. 循环结构与递归的比较

和递归方式相比：

- 没有用到函数调用，每次循环都减少了调用、返回、现场保护等指令。
- 没有使用栈帧，每次循环都节约了栈帧的空间。

由此可见，为了提高程序的性能，若能使用非递归方式，则尽量使用非递归方式实现。



四、过程调用的机器级表示

1. 过程调用的运行机理

例:

```
# c_func.c
```

```
#include <stdio.h>
```

```
int fadd(int x, int y)
```

```
{
```

```
    int u,v,w;
```

```
    u = x + 10;
```

```
    v = y + 25;
```

```
    w = u + v;
```

```
    return w;
```

```
}
```

```
int main()
```

```
{
```

```
    int a = 100;    // 0x 64
```

```
    int b = 200;    // 0x C8
```

```
    int sum = 0;
```

```
    sum = fadd(a, b);
```

```
    printf("%d\n", sum);
```

```
    return 0;
```

```
}
```



1. 过程调用的运行机理

```
13      int  a = 100;    // 0x 64
=> 0x565561ec <+29>:    movl    $0x64,-0x14(%ebp)

14      int  b = 200;    // 0x C8
0x565561f3 <+36>:    movl    $0xc8,-0x10(%ebp)

15      int  sum = 0;
0x565561fa <+43>:    movl    $0x0,-0xc(%ebp)

16      sum = fadd(a, b);
0x56556201 <+50>:    push    -0x10(%ebp)
0x56556204 <+53>:    push    -0x14(%ebp)
0x56556207 <+56>:    call    0x5655619d <fadd>
0x5655620c <+61>:    add     $0x8,%esp
0x5655620f <+64>:    mov     %eax,-0xc(%ebp)
```



1. 过程调用的运行机理

Dump of assembler code for function fadd:

```
3      {
    0x5655619d <+0>:    push    %ebp
    0x5655619e <+1>:    mov     %esp,%ebp
    0x565561a0 <+3>:    sub     $0x10,%esp
    0x565561a3 <+6>:    call   0x56556236 <__x86.get_pc_thunk.ax>
    0x565561a8 <+11>:   add     $0x2e30,%eax

4          int u,v,w;
5          u = x + 10;
=> 0x565561ad <+16>:   mov     0x8(%ebp),%eax
    0x565561b0 <+19>:   add     $0xa,%eax
    0x565561b3 <+22>:   mov     %eax,-0xc(%ebp)

6          v = y + 25;
    0x565561b6 <+25>:   mov     0xc(%ebp),%eax
    0x565561b9 <+28>:   add     $0x19,%eax
    0x565561bc <+31>:   mov     %eax,-0x8(%ebp)
```





1. 过程调用的运行机理

```
7          w = u + v;
0x565561bf <+34>:  mov     -0xc(%ebp),%edx
0x565561c2 <+37>:  mov     -0x8(%ebp),%eax
0x565561c5 <+40>:  add     %edx,%eax
0x565561c7 <+42>:  mov     %eax,-0x4(%ebp)

8          return w;
0x565561ca <+45>:  mov     -0x4(%ebp),%eax

9      }
0x565561cd <+48>:  leave
0x565561ce <+49>:  ret
```



1. 过程调用的运行机理

```
(gdb) p/x $esp
```

```
$2 = 0xffffd170
```

```
(gdb) x/16 0xffffd170-48
```

0xffffd140:	0x00000000	0x00000000	0x01000000	0x0000000b
0xffffd150:	0xf7fc4540	0x00000000	0xf7c184be	0xf7e26054
0xffffd160:	0xf7fbe4a0	0xf7fd6f10	0xf7c184be	0x565561e6
0xffffd170:	0xffffd1b0	0x00000064	0x000000c8	0x00000000

```
(gdb) p/x $eip
```

```
$4 = 0x5655619d
```

```
(gdb) p/x $esp
```

```
$5 = 0xffffd164
```

0xffffd140:	0x00000000	0x00000000	0x01000000	0x0000000b
0xffffd150:	0xf7fc4540	0x00000000	0xf7c184be	0xf7e26054
0xffffd160:	0xf7fbe4a0	0x5655620c	0x00000064	0x000000c8
0xffffd170:	0xffffd1b0	0x00000064	0x000000c8	0x00000000



1. 过程调用的运行机理

0x5655619d: 保护现场，这里只用保护ebp

0x5655619e: 使用ebp，保存进入子程序时，保护现场后堆栈段的基址。EBP = 0xffffd160

0x565561a0: 为局部变量分配空间。三个局部变量，总长度为12个字节。为了对齐，分配了16个字节的空间。

0x565561ad: 源操作数使用了参数x

0x565561b3: 目的操作数使用了局部变量x
注意：

- 参数的寻址形式是：+n(%ebp)
- 局部变量的寻址形式是：-n(%ebp)

```
Dump of assembler code for function fadd:
3      {
    0x5655619d <+0>:  push  %ebp
    0x5655619e <+1>:  mov   %esp,%ebp
    0x565561a0 <+3>:  sub   $0x10,%esp
    0x565561a3 <+6>:  call  0x56556236 <_x86.get_pc_thunk.ax>
    0x565561a8 <+11>: add   $0x2e30,%eax

4          int u,v,w;
5          u = x + 10;
=> 0x565561ad <+16>: mov   0x8(%ebp),%eax
    0x565561b0 <+19>: add   $0xa,%eax
    0x565561b3 <+22>: mov   %eax,-0xc(%ebp)

6          v = y + 25;
    0x565561b6 <+25>: mov   0xc(%ebp),%eax
    0x565561b9 <+28>: add   $0x19,%eax
    0x565561bc <+31>: mov   %eax,-0x8(%ebp)
```



1. 过程调用的运行机理

0x565561ca: 将返回值送入EAX。通过EAX返回子程序的返回值。

0x565561cd: leave指令，等价于

mov %ebp, %esp

pop %ebp

即还原ESP、EBP

0x565561ce: ret, 出栈到EIP

0x5655620c: 出栈参数x、y

```
7          w = u + v;
0x565561bf <+34>:  mov    -0xc(%ebp),%edx
0x565561c2 <+37>:  mov    -0x8(%ebp),%eax
0x565561c5 <+40>:  add    %edx,%eax
0x565561c7 <+42>:  mov    %eax,-0x4(%ebp)

8          return w;
0x565561ca <+45>:  mov    -0x4(%ebp),%eax

9      }
0x565561cd <+48>:  leave

0x565561ce <+49>:  ret
```

1. 过程调用的运行机理

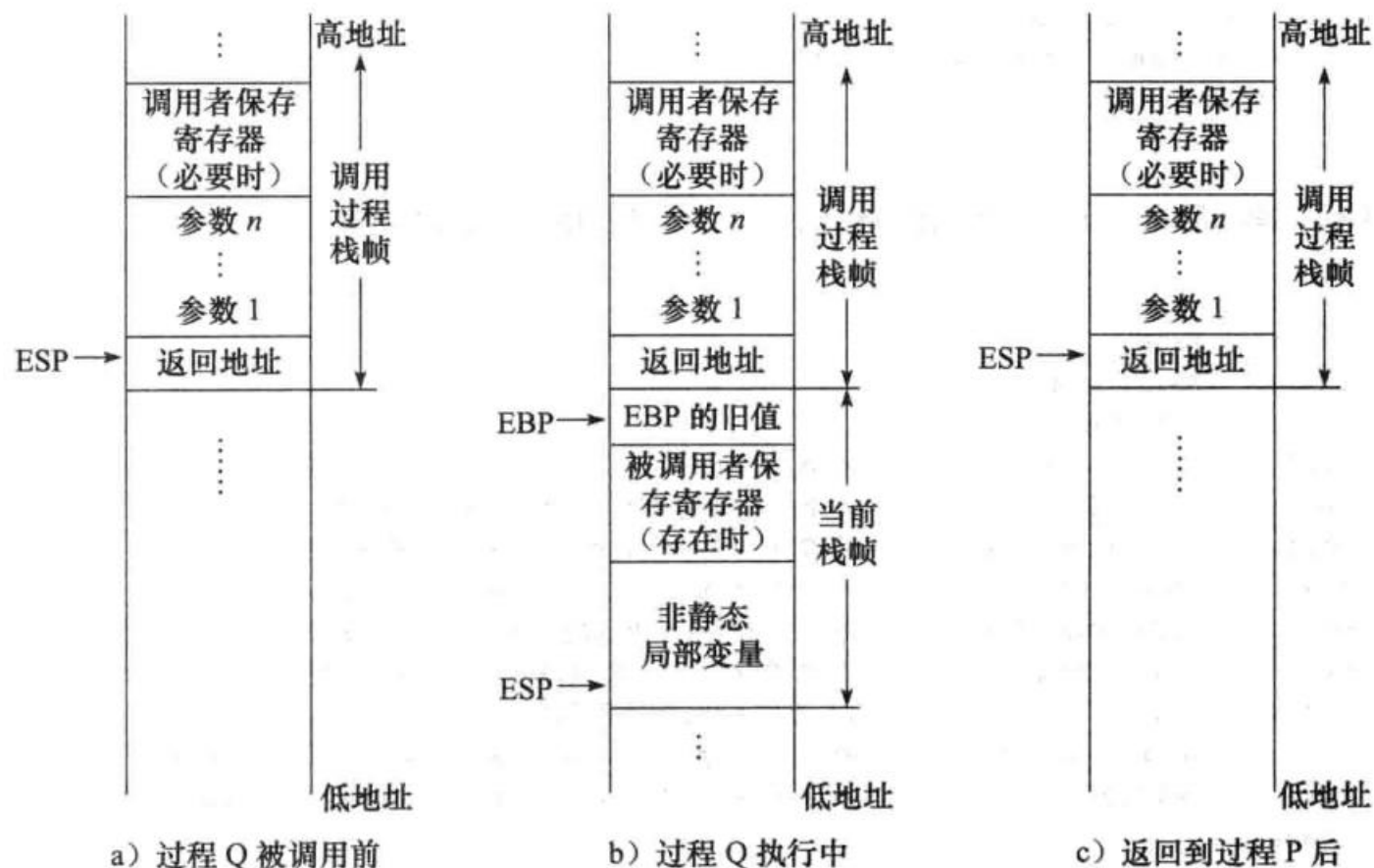


图 3.10 过程调用过程中栈和栈帧的变化

2.局部变量



华中科技大学

- 子程序开始执行时，在堆栈中分配局部变量空间。
- 局部变量在子程序的栈帧中。
- 子程序退出前，会释放局部变量的空间。
- 因此，C语言局部变量的作用域和生存空间，只在该子程序中。





2. 局部变量

例：前例反汇编中

0x565561a0: 可以看到局部变量的分配。

0x565561b3: 目的操作数使用了局部变量x，为 $-0xc(\%ebp)$ 。

0x565561cd: `leave`指令还原了esp的值，即回收了局部变量分配的空间。

Dump of assembler code for function fadd:

```
3      {
    0x5655619d <+0>:  push    %ebp
    0x5655619e <+1>:  mov     %esp,%ebp
    0x565561a0 <+3>:  sub     $0x10,%esp
    0x565561a3 <+6>:  call    0x56556236 <__x86.get_pc_thunk.ax>
    0x565561a8 <+11>: add     $0x2e30,%eax

4          int u,v,w;
5          u = x + 10;
=> 0x565561ad <+16>: mov     0x8(%ebp),%eax
    0x565561b0 <+19>: add     $0xa,%eax
    0x565561b3 <+22>: mov     %eax,-0xc(%ebp)

6          v = y + 25;
    0x565561b6 <+25>: mov     0xc(%ebp),%eax
    0x565561b9 <+28>: add     $0x19,%eax
    0x565561bc <+31>: mov     %eax,-0x8(%ebp)
```

```
7          w = u + v;
    0x565561bf <+34>: mov     -0xc(%ebp),%edx
    0x565561c2 <+37>: mov     -0x8(%ebp),%eax
    0x565561c5 <+40>: add     %edx,%eax
    0x565561c7 <+42>: mov     %eax,-0x4(%ebp)

8          return w;
    0x565561ca <+45>: mov     -0x4(%ebp),%eax

9      }
    0x565561cd <+48>: leave
    0x565561ce <+49>: ret
```

3.按值传递参数和按地址传递参数



华中科技大学

a) 形式参数和实际参数

- 形式参数：在函数定义时出现的参数。形式参数只相当于一个占位符。在编译时，编译检查主函数给出的实际参数是否类型相符。在函数体中，使用形式参数当作变量实现函数体。编译器会将函数体中的形式参数，翻译为堆栈段中的相应的内存操作数，形成完整的汇编指令。





a) 形式参数和实际参数

- 实际参数：函数被调用时给出的参数。实际参数应该和子程序的形式参数一致，并且按照形式参数的顺序（在C语言中，按从右到左的顺序）依次进栈。在程序体中的形式参数翻译成的堆栈段内存操作数，就会正确访问到堆栈段中的实际参数的值。

b) 按值传递参数和按地址传递参数



华中科技大学

- 按值传递参数：主程序调用子程序时，传递的是一个变量的值。即将一个变量的值复制到了堆栈段中。子程序修改该参数时，只是修改了堆栈段中该参数的值，并没有修改主程序中原变量的值。
- 按地址传递参数：主程序调用子程序时，传递的是一个变量的地址。即将一个变量的地址复制到了堆栈段中。子程序可以修改该地址寻址到的操作数的值，即可以修改主程序中原变量的值。



b) 按值传递参数和按地址传递参数



华中科技大学

注意：按地址传递参数，若子程序修改的是堆栈段中的地址参数值，也无法影响主程序中的原变量。只有通过堆栈段中的地址参数值进行间接寻址，修改间接寻址到的操作的值，才能对主程序中的原变量的产生作用。



b) 按值传递参数和按地址传递参数



華中科技大學

例：按值传递参数

```
#include <stdio.h>
```

```
void swap(int x, int y)
```

```
{
```

```
    int t = x;
```

```
    x = y;
```

```
    y = t;
```

```
}
```

```
int main()
```

```
{
```

```
    int a = 15;
```

```
    int b = 22;
```

```
    printf("a=%d b=%d\n", a, b);
```

```
    swap(a, b);
```

```
    printf("a=%d b=%d\n", a, b);
```

```
    return 0;
```

```
}
```





b) 按值传递参数和按地址传递参数

```
14      swap(a, b);  
--Type <RET> for more, q to quit, c to continue without paging--  
=> 0x56556205 <+67>:  sub    $0x8,%esp  
    0x56556208 <+70>:  push   -0xc(%ebp)  
    0x5655620b <+73>:  push   -0x10(%ebp)  
    0x5655620e <+76>:  call   0x5655619d <swap>  
    0x56556213 <+81>:  add    $0x10,%esp
```

```
(gdb) p/x $esp
```

```
$1 = 0xffffd170
```

```
(gdb) x/16 0xffffd170-48
```

0xffffd140:	0xf7e26da0	0x565561a8	0xffffd164	0x00000000
0xffffd150:	0xf7fc4540	0x00000000	0xffffd188	0x56556213
0xffffd160:	0x0000000f	0x00000016	0x00000016	0x565561d9
0xffffd170:	0xffffd1b0	0xf7fbe66c	0x0000000f	0x00000016



b) 按值传递参数和按地址传递参数

```
3      {  
    0x5655619d <+0>:    push    %ebp  
    0x5655619e <+1>:    mov     %esp,%ebp  
    0x565561a0 <+3>:    sub     $0x10,%esp  
    0x565561a3 <+6>:    call    0x5655623d <__x86.get_pc_thunk.ax>  
    0x565561a8 <+11>:   add     $0x2e30,%eax  
  
4      int t = x;  
=> 0x565561ad <+16>:   mov     0x8(%ebp),%eax  
    0x565561b0 <+19>:   mov     %eax,-0x4(%ebp)  
  
5      x = y;  
    0x565561b3 <+22>:   mov     0xc(%ebp),%eax  
    0x565561b6 <+25>:   mov     %eax,0x8(%ebp)  
  
6      y = t;  
    0x565561b9 <+28>:   mov     -0x4(%ebp),%eax  
    0x565561bc <+31>:   mov     %eax,0xc(%ebp)  
  
7      }  
    0x565561bf <+34>:   nop  
    0x565561c0 <+35>:   leave  
    0x565561c1 <+36>:   ret
```



b) 按值传递参数和按地址传递参数



華中科技大學

例：按地址传递参数

```
#include <stdio.h>
void swap(int* x, int* y)
{
    int t = *x;
    *x = *y;
    *y = t;
}
```

```
int main()
{
    int a = 15;
    int b = 22;
    printf("a=%d b=%d\n", a, b);
    swap(&a, &b);
    printf("a=%d b=%d\n", a, b);
    return 0;
}
```





b) 按值传递参数和按地址传递参数

```
14      swap(&a, &b);  
=> 0x5655622a <+80>:  sub    $0x8,%esp  
    0x5655622d <+83>:  lea     -0x10(%ebp),%eax  
    0x56556230 <+86>:  push   %eax  
    0x56556231 <+87>:  lea     -0x14(%ebp),%eax  
    0x56556234 <+90>:  push   %eax  
    0x56556235 <+91>:  call   0x565561ad <swap>  
    0x5655623a <+96>:  add     $0x10,%esp
```

```
(gdb) p/x $esp
```

```
$2 = 0xffffd170
```

```
(gdb) x/16 0xffffd170-48
```

0xffffd140:	0xf7e26da0	0x565561b8	0xffffd164	0x00000000
0xffffd150:	0xf7fc4540	0x00000000	0xffffd188	0x5655623a
0xffffd160:	<u>0xffffd174</u>	<u>0xffffd178</u>	0x00000016	<u>0x565561f1</u>
0xffffd170:	<u>0xffffd1b0</u>	0x0000000f	0x00000016	0x79c58000



b) 按值传递参数和按地址传递参数

```
3      {
0x565561ad <+0>:    push    %ebp
0x565561ae <+1>:    mov     %esp,%ebp
0x565561b0 <+3>:    sub     $0x10,%esp
0x565561b3 <+6>:    call   0x56556277 <__x86.get_pc_thunk.ax>
0x565561b8 <+11>:   add     $0x2e1c,%eax

4      int t = *x;
=> 0x565561bd <+16>:  mov     0x8(%ebp),%eax
0x565561c0 <+19>:  mov     (%eax),%eax
0x565561c2 <+21>:  mov     %eax,-0x4(%ebp)

5      *x = *y;
0x565561c5 <+24>:  mov     0xc(%ebp),%eax
0x565561c8 <+27>:  mov     (%eax),%edx
0x565561ca <+29>:  mov     0x8(%ebp),%eax
0x565561cd <+32>:  mov     %edx,(%eax)

6      *y = t;
0x565561cf <+34>:  mov     0xc(%ebp),%eax
0x565561d2 <+37>:  mov     -0x4(%ebp),%edx
0x565561d5 <+40>:  mov     %edx,(%eax)

7      }
0x565561d7 <+42>:  nop
0x565561d8 <+43>:  leave
0x565561d9 <+44>:  ret
```





4. 递归过程调用

例：使用递归子程序，求 $N!$

```
#include <stdio.h>
```

```
int f(int x)
```

```
{
```

```
    if (x==1)
```

```
        return 1;
```

```
    return x*f(x-1);
```

```
}
```

```
int main()
```

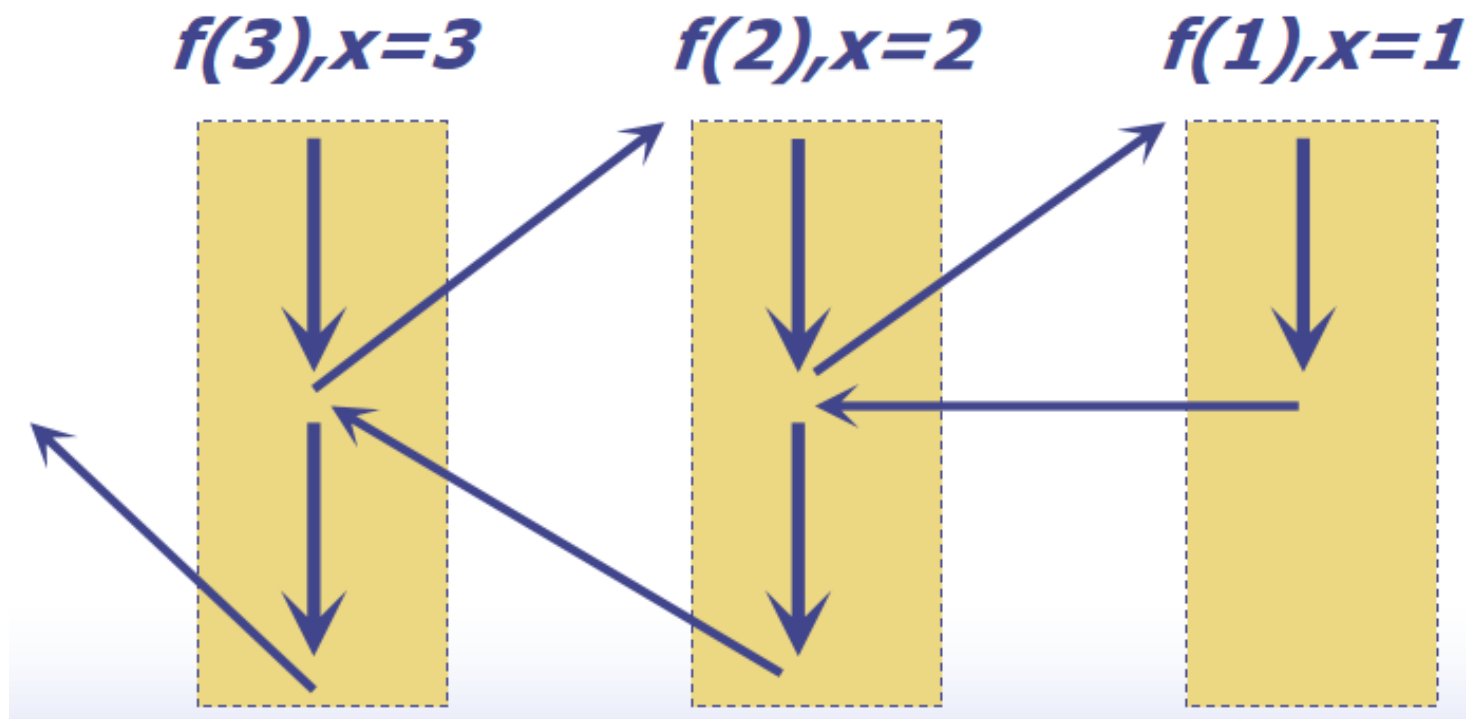
```
{
```

```
    printf("%d\n",f(5));
```

```
    return 0;
```

```
}
```

4. 递归过程调用





4. 递归过程调用

```
3      {
    0x5655619d <+0>:    push    %ebp
    0x5655619e <+1>:    mov     %esp,%ebp
    0x565561a0 <+3>:    sub     $0x8,%esp
    0x565561a3 <+6>:    call   0x5655621b <__x86.get_pc_thunk.ax>
    0x565561a8 <+11>:   add     $0x2e30,%eax

4          if (x==1)
=> 0x565561ad <+16>:   cmpl    $0x1,0x8(%ebp)
    0x565561b1 <+20>:   jne     0x565561ba <f+29>

5          return 1;
    0x565561b3 <+22>:   mov     $0x1,%eax
    0x565561b8 <+27>:   jmp     0x565561d0 <f+51>

6          return x*f(x-1);
    0x565561ba <+29>:   mov     0x8(%ebp),%eax
    0x565561bd <+32>:   sub     $0x1,%eax
    0x565561c0 <+35>:   sub     $0xc,%esp
    0x565561c3 <+38>:   push    %eax
    0x565561c4 <+39>:   call   0x5655619d <f>
    0x565561c9 <+44>:   add     $0x10,%esp
    0x565561cc <+47>:   imul    0x8(%ebp),%eax

7      }
    0x565561d0 <+51>:   leave
    0x565561d1 <+52>:   ret
```

